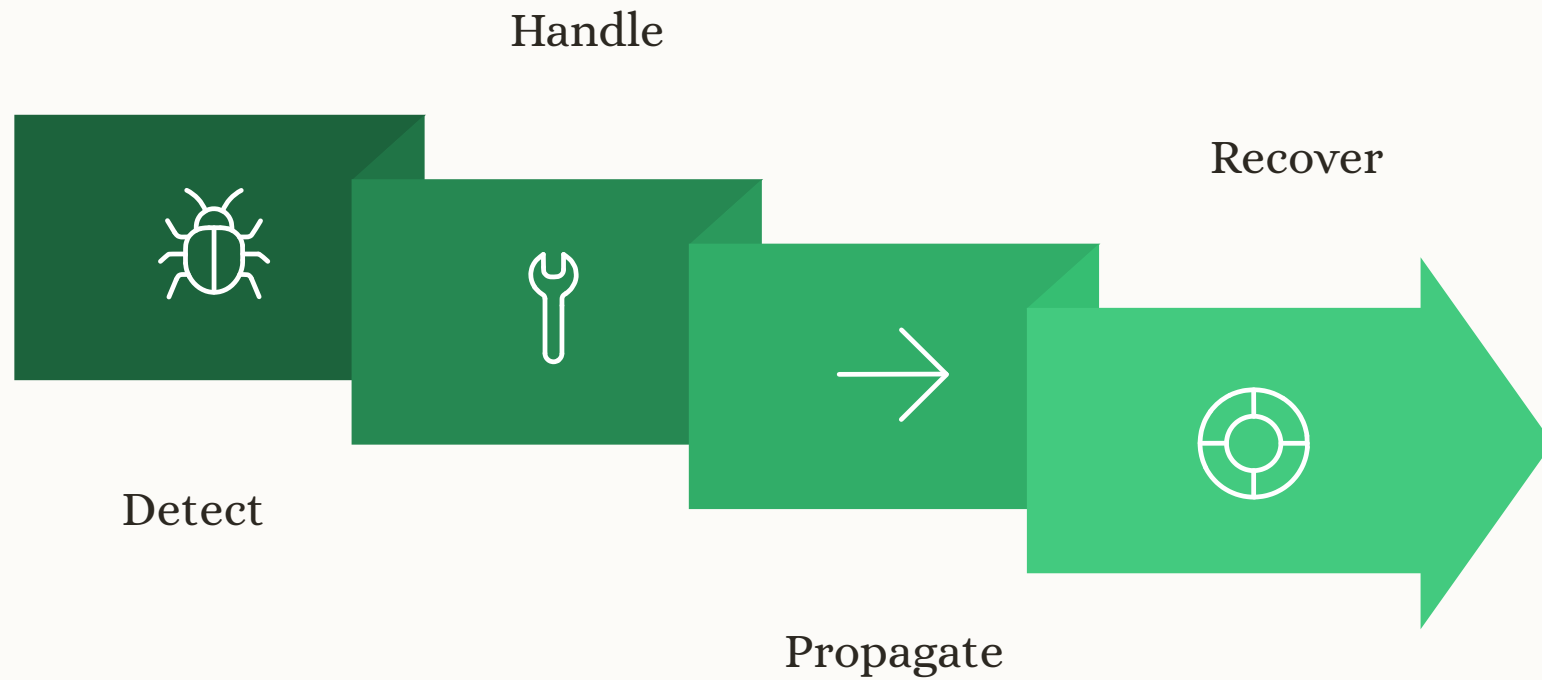


Error Handling with try...catch Blocks

A Comprehensive Guide to Managing Errors in Synchronous and Asynchronous Programming



Introduction to Error Handling

What is an Error in Programming?

An error is an unexpected event that disrupts the normal flow of programme execution. Errors can occur due to invalid user input, network failures, resource unavailability, or logical mistakes in code. When unhandled, errors cause programmes to crash or behave unpredictably, leading to poor user experience and system instability.

Errors are broadly classified into three categories: syntax errors (detected during compilation), runtime errors (occurring during execution), and logical errors (producing incorrect results without crashing).

Why Proper Error Handling Matters

Effective error handling ensures programme robustness and reliability. It prevents abrupt termination, provides meaningful feedback to users, and facilitates debugging by logging error information. Proper error management improves code maintainability and enables graceful degradation when components fail.

In production environments, unhandled errors can lead to data corruption, security vulnerabilities, and system downtime. Implementing structured error handling mechanisms is therefore essential for building professional-grade applications.



try...catch Syntax and Structure

try Block

Contains code that might throw an error. Execution begins here and continues until an error occurs or the block completes successfully.

catch Block

Executes only if an error is thrown in the try block. Receives the error object as a parameter for inspection and handling.

finally Block

Executes regardless of whether an error occurred. Used for cleanup operations like closing files or releasing resources.

Basic Syntax Example

```
try {
  // Code that may throw an error
  let result = riskyOperation();
  console.log(result);
} catch (error) {
  // Handle the error
  console.error("An error occurred:", error.message);
} finally {
  // Always executes
  console.log("Cleanup completed");
}
```

The catch block receives an error object containing properties such as `name`, `message`, and `stack`. The finally block is optional but useful for ensuring critical cleanup code executes regardless of success or failure. This structure provides a clean separation between normal execution flow and error handling logic.



Error Handling in Asynchronous Code

Synchronous vs Asynchronous Error Handling

Synchronous error handling uses traditional try...catch blocks where errors are thrown and caught immediately within the same call stack. The programme execution is sequential, making error tracking straightforward.

Asynchronous error handling requires special consideration because operations complete at unpredictable times. Errors in callbacks or promises may occur outside the original try...catch scope, necessitating different handling strategies.

Using try...catch with async/await

```
async function fetchData() {
  try {
    const response = await fetch(url);
    const data = await response.json();
    return data;
  } catch (error) {
    console.error("Fetch failed:", error);
    throw error;
  }
}
```

1

Promise Rejection

Unhandled promise rejections occur when async operations fail without catch handlers

2

async/await Syntax

Makes asynchronous code appear synchronous, allowing try...catch to work naturally

3

Error Propagation

Errors can be re-thrown to allow higher-level handlers to manage them

The async/await syntax introduced in ES2017 revolutionised asynchronous error handling by enabling traditional try...catch blocks to catch promise rejections. This approach provides cleaner, more readable code compared to chaining .catch() methods on promises.



Common Errors and Best Practices

1	2	3
Runtime Errors TypeError: Attempting operations on incompatible data types ReferenceError: Accessing undefined variables RangeError: Numeric values outside valid range	Network & Promise Errors Network failures: Connection timeouts or server unavailability Promise rejections: Asynchronous operations that fail API errors: Invalid responses or authentication failures	Best Practices Specific error handling: Catch and handle different error types appropriately Logging: Record errors with context for debugging User feedback: Provide meaningful messages without exposing sensitive details

Key Learning Outcomes

- try...catch blocks provide structured error handling mechanism
- finally block ensures cleanup code always executes
- async/await enables try...catch usage in asynchronous code
- Proper error handling prevents programme crashes and improves reliability
- Error objects contain name, message, and stack trace properties
- Different error types require tailored handling strategies



Exam Focus Points

2-mark questions: Define try...catch syntax, explain finally block purpose

5-mark questions: Compare synchronous vs asynchronous error handling with code examples